based on my project report explain the code train.py file and its other inter linked files which are used and executed when python train.py command is runned i need complete flow of the execution when it is runned along with in depth explaination of each flow Uploaded files: - **sonar_ship_detection_-_Copy__2_.zip** (201.6 MB)

I've read train.py and every file it touches, and I'm writing up the full walkthrough now.

Two caveats first:

- **No project report was in the zip.** This explanation comes only from the code. If you paste the relevant report sections, I can match the wording and chapter structure.
- **The script that creates data/train.npz and data/val.npz is missing from the zip.** train.py needs those files, and there is no data/ folder either. simulator.py and simulator_v2.py provide the building blocks for generating them, but nothing in the zip calls them to write the splits.

1. Which files take part

When you run python train.py, only three project files are actually used:

File	Role	When it runs
train.py	The training script itself	Executed top to bottom
sonar_config.py	Paths, constants, device selection	Imported at line 8
sonar_model.py	Builds ResNet18 or EfficientNet-B0	Imported at line 9

External inputs and outputs:

- **Input:** data/train.npz and data/val.npz, which hold precomputed mel-spectrograms.
- **Outputs:** checkpoints/<arch>_best.pt and results/history_<arch>.json.

Files that are related but not executed by train.py:

- simulator.py and simulator_v2.py create the synthetic sonar data upstream.
- real_prepare.py and real_split.py prepare the real ShipsEar data.

- infer.py and sonar_app.py load the checkpoint that train.py saves.
- train_mixed.py does from train import augment, so it reuses train.py's augmentation.

2. Execution flow

```
python train.py
  │
  ├─ [A] Interpreter start-up: set env var, import libraries
  ├─ [B] import sonar_config   → runs sonar_config.py (constants, helper functions)
  ├─ [C] import sonar_model    → runs sonar_model.py (defines model builders)
  ├─ [D] Define load_split, augment, evaluate, main (nothing runs yet)
  └─ [E] __name__ == "__main__" → main()
      ├─ 1. Parse CLI arguments
      ├─ 2. Setup: make_dirs, seeds, device, AMP flag
      ├─ 3. Load train/val .npz onto the device
      ├─ 4. build_model → ResNet18 with 1-channel input and 1 output logit
      ├─ 5. Loss, optimizer, LR scheduler, grad scaler
      ├─ 6. EPOCH LOOP
      │    ├─ shuffle → mini-batches → augment → forward → loss
      │    ├─ backward → optimizer step → scheduler step
      │    ├─ evaluate on validation set
      │    └─ save checkpoint if val loss improved / early stop
      └─ 7. Save history JSON, print best epoch
```

Stage A: start-up (lines 1–9)

- **Line 2** sets PYTORCH_ENABLE_MPS_FALLBACK=1 using setdefault, so a value you set yourself is not overwritten. It must happen before import torch because PyTorch reads it at load time. On Apple Silicon (MPS) it lets operations the GPU backend doesn't support fall back to the CPU instead of crashing.
- **Lines 4–7** import the standard libraries, NumPy, PyTorch and torch.nn.

Stage B: import sonar_config as config

Python executes all of sonar_config.py at this point. It does not create any folders yet; it only defines values.

- **Paths:** ROOT is the folder containing the file, found with Path(__file__).resolve().parent. DATA_DIR, CKPT_DIR and RESULTS_DIR are subfolders of it.

- **Signal settings:** the sample rate is 8000 Hz and each clip lasts 5 s, so N_SAMPLES = 40000.
- **Spectrogram settings:** N_FFT=1024, HOP_LENGTH=313, N_MELS=128, with a 20–4000 Hz range.
- **Dataset split:** NUM_SAMPLES=10000 split 70/15/15.
- **Training defaults:** BATCH_SIZE=64, EPOCHS=20, LEARNING_RATE=1e-3.
- **Functions:** get_device() and make_dirs() are defined here and called later.

The spectrogram settings explain the input size. 40000 samples with a hop of 313 give 1 + 40000//313 = 128 time frames. Each clip is therefore a 128 × 128 image: 128 mel bands by 128 time steps.

Stage C: from sonar_model import build_model, count_params

This runs sonar_model.py, which imports torchvision.models and defines _build, build_model and count_params. No model is created yet.

Stage D: function definitions

Python reads load_split, augment, evaluate and main and stores them as function objects. Nothing inside them executes until main() is called.

Stage E: main()

Step 1: argument parsing (lines 48–58)

Argument	Default	Meaning
--arch	resnet18	resnet18 or efficientnet_b0
--epochs	20 (from config)	Maximum epochs
--batch	64	Batch size
--lr	1e-3	Peak learning rate
--patience	5	Early-stopping patience
--no-pretrained	off	Train from scratch instead of using ImageNet weights
--train-file / --val-file	train / val	Which .npz files to load from data/
--ckpt-dir	checkpoints	Where to save the model

The last three arguments are why the same script can train on different datasets, such as a v2_train set, and write to checkpoints_v2/. I'm inferring that this is how checkpoints_v2 was produced; the zip doesn't show the commands.

Step 2: setup (lines 60–64)

- config.make_dirs() creates data/, checkpoints/ and results/ if they are missing.
- torch.manual_seed(42) and np.random.seed(42) make weight initialisation and shuffling reproducible. Some GPU operations can still be non-deterministic.
- get_device() picks CUDA first, then MPS (Apple), then CPU.
- use_amp = device.type == "cuda". Mixed-precision training is only turned on for NVIDIA GPUs.

Step 3: data loading (load_split, lines 12–16)

np.load("data/train.npz") is read, and the arrays are used as follows:

- X has shape (N, 128, 128) and is stored as float16, which halves memory. It is converted to a tensor and moved to the device.
- y holds the labels, 0 for ocean noise only and 1 for ship present. It is converted to float32 because the loss function needs float targets.

The whole dataset sits on the GPU, so there is no DataLoader and no per-batch CPU-to-GPU copying. That makes training fast, and 7000 spectrograms at 128 × 128 in float16 take only about 230 MB. This is also why NUM_WORKERS in the config is not used by this script.

Step 4: model creation (sonar_model.py)

build_model("resnet18", pretrained=True) calls _build:

1. It loads torchvision's ResNet18 with ImageNet weights. The first time, these are downloaded and cached.
2. **The first layer is replaced.** ImageNet models expect 3-channel RGB, but a spectrogram has 1 channel. A new Conv2d(1, 64, 7×7, stride 2) is created, and its weights are the **mean of the old RGB filters**. This keeps the pretrained edge and texture detectors useful.
3. **The classifier is replaced.** fc becomes Linear(512, 1), so the network outputs one raw logit for ship vs. no ship.
4. If the weight download fails, build_model catches the error, prints a warning and builds the model from scratch.

For EfficientNet-B0 the same pattern applies: the first conv in features[0][0] is replaced, and the classifier becomes Dropout(0.2) plus Linear(1280, 1).

For a 128 × 128 input, ResNet18's feature map shrinks 64 → 32 → 16 → 8 → 4. Global average pooling then reduces it to 512 values, and the final layer produces 1 logit. The printed parameter count is about 11.2M for ResNet18 and about 4.0M for EfficientNet-B0.

Step 5: training components (lines 73–82)

- **Loss:** BCEWithLogitsLoss applies a sigmoid internally and is numerically stable. It expects raw logits, which is why the model has no sigmoid layer.
- **Optimizer:** AdamW with weight decay 1e-4.
- **Scheduler:** OneCycleLR over epochs × steps_per_epoch steps. The learning rate starts at about max_lr/25, rises to 1e-3 over the first 30% of training, then decays with a cosine curve to nearly zero. It is stepped after every batch, not every epoch.
- **GradScaler:** active only on CUDA. It scales the loss to prevent float16 gradient underflow.
- **Tracking variables:** history, best_val = inf and bad = 0 (the counter of epochs without improvement).

Step 6: the epoch loop (lines 85–122)

For each epoch:

1. **Shuffle.** perm = randperm(N) is a fresh random order on the device.
2. **Batch.** Indices are sliced perm[i:i+64]. A final batch with fewer than 2 samples is skipped, because BatchNorm cannot compute statistics from one sample.
3. **Augment.** Xtr[idx].unsqueeze(1).float() gives shape (B, 1, 128, 128). augment then applies two random operations to each sample:
    - **Circular time shift:** a random shift is chosen per sample. cols = (arange(W) − shift) % W and gather along the time axis rotate each spectrogram. The ship signature is a stationary pattern, so where it starts in time carries no information.
    - **Time masking:** with 50% probability, a random block of 4–16 time frames is set to 0. Because each spectrogram was standardised to mean 0, zero is the average level, so this simulates a short dropout or burst. It is similar in spirit to SpecAugment.

    Augmentation is applied only to training data, never to validation.

4. **Forward.** zero_grad(set_to_none=True) clears gradients. Inside autocast, model(xb) gives (B, 1), which is squeezed to (B,). The loss is computed on out.float() for stability.

5. **Backward and update.**

    - scaler.scale(loss).backward() computes gradients.
    - scaler.step(opt) unscales them and updates the weights.
    - scaler.update() adjusts the scale factor.
    - sched.step() moves the learning-rate schedule forward.
    - On CPU or MPS the scaler is disabled, so these calls behave like plain backward() and step().

6. **Running metrics.** Loss is accumulated weighted by batch size. A prediction counts as correct when logit > 0, which is the same as probability > 0.5. Note that training accuracy is measured on augmented data while the model is in train mode, so it is usually slightly lower than the validation accuracy early on.

7. **Validation (evaluate).**

    - model.eval() fixes BatchNorm to its running statistics and turns dropout off.
    - @torch.no_grad() skips the gradient bookkeeping.
    - The validation data is processed in batches of 256, and the loss and accuracy are averaged over the whole set.

8. **Logging.** The epoch row is appended to history and printed with its time.

9. **Checkpointing and early stopping.**

    - If val_loss < best_val, the counter resets and a dict is saved to checkpoints/resnet18_best.pt. It contains arch, state_dict, epoch, val_loss and val_acc.
    - Otherwise bad increases, and when bad >= 5 training stops.

The checkpoint is chosen by lowest validation loss, not highest accuracy. Loss is a smoother signal and reflects how confident the model is, which makes it a more stable criterion.

Step 7: wrap-up (lines 124–128)

- The history is written to results/history_resnet18.json.

- min(history, key=val_loss) finds the best epoch, and the script prints that epoch, its validation loss and accuracy, and the checkpoint path.

3. How the output is used afterwards

- **infer.py** calls load_model, which reads config.CKPT_DIR/<arch>_best.pt and rebuilds the model with build_model(arch, pretrained=False). It then calls load_state_dict. The architecture must match what train.py saved. Each 5 s window goes through sim.to_spectrogram, the same pipeline that produced the training data, and then sigmoid(model(x)) gives P(ship).
- **sonar_app.py** (the PyQt6 GUI) uses the same model-building and spectrogram code. Its error message "No checkpoints in 'checkpoints/'. Run train.py first." confirms train.py is the entry point that produces the model.
- **train_mixed.py** loads checkpoints_v2/resnet18_best.pt, then fine-tunes it on a mix of synthetic and real ShipsEar data. It reuses augment from train.py and adds its own spec_aug.

4. How the spectrograms are made (upstream of train.py)

The X arrays in the .npz files come from simulator.py's to_spectrogram:

1. A mel-spectrogram with power 2.0.
2. Conversion to decibels, with an 80 dB dynamic range.
3. Per-sample standardisation to mean 0 and standard deviation 1.

That per-sample normalisation is why masking with 0 in augment is a neutral value.

simulator.py synthesises class 0 (coloured ocean noise, plus hard negatives such as whale chirps, clicks and tones) and class 1 (ocean noise plus a ship signature built from harmonics, DEMON blade-rate modulation, cavitation noise and Doppler shift, mixed at an SNR between −15 and +5 dB). simulator_v2.py makes this harder by adding random frequency shaping, impulsive noise and more varied ship types, with SNR down to −20 dB.

If you'd like, I can turn this into a Word or PDF document for your report. If you paste the report text, I can also align the terminology with it, or add a diagram of the flow.

now explain each line by line code what exactly that line is doing in depth explaination regarding each line

Line-by-line walkthrough of train.py

Line numbers match the file as uploaded. Where a line calls into sonar_config.py or sonar_model.py, I explain what happens there too.

Part 1: Imports and setup (lines 1–9)

```
1  import os
2  os.environ.setdefault("PYTORCH_ENABLE_MPS_FALLBACK", "1")   # must be set before torch loads
```

- **Line 1** imports the os module, which gives access to environment variables.
- **Line 2** adds the variable PYTORCH_ENABLE_MPS_FALLBACK=1 to the process environment. setdefault only sets it if it doesn't already exist, so a value you set in your terminal is not overwritten.
- PyTorch reads this variable when it loads, so it must be set before import torch on line 6. On Apple Silicon (the MPS GPU backend), it lets any operation MPS doesn't support run on the CPU instead of raising an error. On Windows or NVIDIA machines it does nothing.

```
4  import argparse, json, math, time
```

Four standard-library modules, imported on one line:

- argparse reads command-line options such as --arch.
- json writes the training history to a file.
- math is used for math.ceil when computing steps per epoch.
- time measures how long each epoch takes.

```
5  import numpy as np
6  import torch
7  import torch.nn as nn
```

- **Line 5** imports NumPy for loading .npz files and seeding.
- **Line 6** imports PyTorch, which provides tensors, GPU support, autograd and optimizers.
- **Line 7** imports the neural-network module under the short name nn, which gives access to nn.BCEWithLogitsLoss.

```
8  import sonar_config as config
```

This runs sonar_config.py once and stores its names under config. That file imports Path and torch and defines:

- the paths ROOT, DATA_DIR, CKPT_DIR and RESULTS_DIR;
- the constants SAMPLE_RATE=8000, DURATION=5.0 and N_SAMPLES=40000;
- the spectrogram settings and the training defaults;
- the functions get_device() and make_dirs().

Nothing is created on disk at this point.

```
9  from sonar_model import build_model, count_params
```

This runs sonar_model.py, which imports torchvision.models and defines _build, build_model and count_params. Only those two names are brought into train.py. No model exists yet.

Part 2: load_split (lines 12–16)

```
12  def load_split(name, device):
```

Defines a function that takes a split name ("train" or "val") and a device.

```
13      d = np.load(config.DATA_DIR / f"{name}.npz")
```

- config.DATA_DIR / f"{name}.npz" uses Path's / operator to build the path .../data/train.npz.
- np.load opens the compressed archive. It does not read everything immediately; it returns a lazy dictionary-like object. The arrays are only read when you index them, as d["X"] does below.

```
14      X = torch.from_numpy(d["X"]).to(device)          # float16, kept on device
```

- d["X"] reads the spectrogram array, with shape (N, 128, 128) and dtype float16.
- torch.from_numpy wraps it as a tensor without copying.
- .to(device) copies it onto the GPU if there is one. The whole dataset lives on the device, so no per-batch transfer is needed later.
- Float16 halves the storage cost. It is converted to float32 when used.

```
15      y = torch.from_numpy(d["y"]).float().to(device)
```

Loads the labels (0 or 1), converts them to float32 because BCEWithLogitsLoss needs float targets, and moves them to the device.

```
16      return X, y
```

Returns both tensors.

Part 3: augment (lines 19–31)

```
19 def augment(x):
20     """x: (B,1,H,W). Random circular time shift + random time-frame mask."""
```

The input is a batch of spectrograms, shape (batch, 1 channel, 128 mel bins, 128 time frames). The docstring documents that.

```
21     B, _, H, W = x.shape
22     dev = x.device
```

Unpacks the shape. `B` is the batch size, `_` ignores the channel count, `H` is the number of mel bins (frequency) and `W` is the number of time frames. `dev` is saved so new tensors are created on the same device.

Circular time shift (lines 23–25)

```
23     shifts = torch.randint(0, W, (B,), device=dev)
```

Creates one random integer per sample, between 0 and $W-1$ (the upper bound is exclusive). This is how far each sample will be rotated in time.

```
24     cols = (torch.arange(W, device=dev)[None, :] - shifts[:, None]) % W
```

This builds, for each sample, the list of source columns for the rotated output.

- `torch.arange(W)[None, :]` has shape (1, W) and holds 0, 1, ..., 127.
- `shifts[:, None]` has shape (B, 1).
- Subtracting broadcasts to (B, W). Entry (b, j) is j − shift_b.
- `% W` wraps negative numbers around, so −3 becomes 125. This is what makes the shift circular.

```
25     x = x.gather(3, cols[:, None, None, :].expand(-1, 1, H, -1))
```

- `cols[:, None, None, :]` reshapes the indices to (B, 1, 1, W).
- `.expand(-1, 1, H, -1)` repeats them across all H frequency rows to give (B, 1, H, W). `expand` is a view, not a real copy.
- `x.gather(3, index)` works along dimension 3, the time axis. The result is `out[b, 0, h, j] = x[b, 0, h, index[b, 0, h, j]]`. Every frequency row of a sample is rotated by the same amount, so the spectrogram's structure is preserved.

Random time masking (lines 27–31)

```
27      start = torch.randint(0, W, (B,), device=dev)
28      width = torch.randint(4, 17, (B,), device=dev)
```

For each sample, a random start frame (0–127) and a random mask width. The upper bound 17 is exclusive, so widths are 4 to 16 frames.

```
29      t = torch.arange(W, device=dev)[None, :]
```

The time-frame indices 0..127, shape (1, W).

```
30      mask = (t >= start[:, None]) & (t < (start + width)[:, None]) & (torch.rand(B,
device=dev) < 0.5)[:, None]
```

This combines three conditions into a boolean mask of shape (B, W):

1. t >= start: frames at or after the start.
2. t < start + width: frames before the end of the block.
3. torch.rand(B) < 0.5: a coin flip per sample, so only about half the samples are masked.

A frame is masked only if all three are true. A mask near the right edge is simply cut off and does not wrap around.

```
31      return x.masked_fill(mask[:, None, None, :], 0.0)
```

mask[:, None, None, :] becomes (B, 1, 1, W) and broadcasts across the channel and frequency axes. Wherever it is True, the value is replaced by 0.0. Because each spectrogram was standardised to mean 0, zero means “average energy,” not “silence.” The function returns the augmented batch.

Part 4: evaluate (lines 34–44)

```
34  @torch.no_grad()
35  def evaluate(model, X, y, loss_fn, bs=256):
```

- The decorator turns off gradient tracking for the whole function, which saves memory and time because nothing is being trained here.
- The parameters are the model, the data and labels, the loss function and a batch size of 256 (larger than training because no gradients are stored).

```
36      model.eval()
```

Switches layers to inference behaviour. BatchNorm uses its stored running statistics instead of the current batch’s, and Dropout (in EfficientNet) is disabled.

```
37      total_loss, correct = 0.0, 0
```

Two accumulators for the summed loss and the count of correct predictions.

```
38      for i in range(0, len(X), bs):
```

Steps through the data in chunks: 0, 256, 512, and so on.

```
39          xb = X[i:i + bs].unsqueeze(1).float()
40          yb = y[i:i + bs]
```

- **Line 39** slices a chunk, shape (≤256, 128, 128), adds the channel dimension to get (≤256, 1, 128, 128), and converts float16 to float32.
- **Line 40** slices the matching labels.

```
41          out = model(xb).squeeze(1)
```

The forward pass gives (B, 1) logits, and `squeeze(1)` flattens them to (B,) to match the labels.

```
42          total_loss += loss_fn(out, yb).item() * len(yb)
```

`loss_fn` returns the **mean** loss for the chunk. Multiplying by `len(yb)` turns it back into a sum, so chunks of different sizes (the last one is usually smaller) are weighted correctly. `.item()` converts the one-element tensor to a Python float.

```
43          correct += ((out > 0).float() == yb).sum().item()
```

- `out > 0` is the prediction: a logit above 0 means probability above 0.5, so "ship."
- `.float()` converts the booleans to 0.0 or 1.0.
- `== yb` compares with the true labels, `.sum()` counts the matches, and `.item()` converts to an int.

```
44      return total_loss / len(X), correct / len(X)
```

Returns the average loss and the accuracy over the whole validation set.

Part 5: `main()` setup (lines 47–82)

```
47  def main():
48      ap = argparse.ArgumentParser()
```

Creates the command-line parser.

```
49    ap.add_argument("--arch", default="resnet18", choices=["resnet18",
"efficientnet_b0"])
```

Defines --arch. If you don’t pass it, it is "resnet18". Any other value is rejected with a clear error message.

```
50    ap.add_argument("--epochs", type=int, default=config.EPOCHS)
51    ap.add_argument("--batch", type=int, default=config.BATCH_SIZE)
52    ap.add_argument("--lr", type=float, default=config.LEARNING_RATE)
```

Define the epoch count (default 20), batch size (default 64) and peak learning rate (default 0.001). type= converts the text you type into a number. The defaults come from sonar_config.py, so the configuration lives in one place.

```
53    ap.add_argument("--patience", type=int, default=5)
```

How many epochs in a row without improvement are allowed before stopping.

```
54    ap.add_argument("--no-pretrained", action="store_true")
```

A flag with no value. If you write --no-pretrained it becomes True, otherwise False. It is accessed later as args.no_pretrained because dashes become underscores.

```
55    ap.add_argument("--train-file", default="train")
56    ap.add_argument("--val-file", default="val")
57    ap.add_argument("--ckpt-dir", default="checkpoints")
```

These choose which .npz files in data/ to load and which folder to save the checkpoint in. They are what let you train on other datasets, for example the v2 data, and write to checkpoints_v2.

```
58    args = ap.parse_args()
```

Reads sys.argv. For plain python train.py, every option takes its default. The values are available as args.arch, args.epochs and so on.

```
60    config.make_dirs()
```

Calls make_dirs() in sonar_config.py. It loops over DATA_DIR, CKPT_DIR and RESULTS_DIR and runs mkdir(parents=True, exist_ok=True) on each. Missing folders are created, and existing ones cause no error.

```
61    torch.manual_seed(config.SEED)
62    np.random.seed(config.SEED)
```

Seeds PyTorch's and NumPy's random number generators with 42. Weight initialisation of new layers and randperm shuffling become repeatable. Some CUDA operations can still be non-deterministic.

```
63      device = config.get_device()
```

Calls get_device(), which returns CUDA if an NVIDIA GPU is available, otherwise MPS if an Apple GPU is available, otherwise CPU.

```
64      use_amp = device.type == "cuda"
```

A boolean that is True only on CUDA. Automatic mixed precision (float16 math) is used only on NVIDIA GPUs.

```
66      Xtr, ytr = load_split(args.train_file, device)
67      Xva, yva = load_split(args.val_file, device)
```

Loads data/train.npz and data/val.npz onto the device using Part 2.

```
69      model = build_model(args.arch, pretrained=not args.no_pretrained).to(device)
```

- not args.no_pretrained is True by default, so ImageNet weights are used.
- build_model calls _build, which:
    1. loads torchvision's ResNet18 with ImageNet weights;
    2. replaces conv1 with a 1-channel convolution whose weights are the mean of the RGB filters;
    3. replaces fc with Linear(512, 1).
- If loading the weights fails, for example with no internet, it prints a warning and builds the model from scratch.
- .to(device) moves all of the model's parameters onto the GPU.

```
70      print(f"Arch: {args.arch} | params: {count_params(model)/1e6:.1f}M | device:
{device} | "
71            f"train {len(Xtr)} / val {len(Xva)}")
```

Prints a summary line. Two adjacent string literals are joined automatically. count_params sums p.numel() over all parameters, and dividing by 1e6 with :.1f shows it in millions with one decimal place.

```
73      loss_fn = nn.BCEWithLogitsLoss()
```

Binary cross-entropy that takes raw logits. It applies the sigmoid and the log together in a numerically stable way, so the model needs no sigmoid layer.

```
74      opt = torch.optim.AdamW(model.parameters(), lr=args.lr, weight_decay=1e-4)
```

Creates the AdamW optimizer for all the model's parameters. weight_decay=1e-4 applies a small decoupled shrinkage to the weights each step, which acts as regularisation.

```
75      steps = math.ceil(len(Xtr) / args.batch)
```

Batches per epoch, rounded up so the final partial batch is counted. For example, 7000 / 64 = 109.4, which becomes 110.

```
76      sched = torch.optim.lr_scheduler.OneCycleLR(opt, max_lr=args.lr,
epochs=args.epochs, steps_per_epoch=steps)
```

Creates the learning-rate schedule over epochs × steps total updates. With the defaults:

- It starts at max_lr / 25.
- It rises to max_lr over the first 30% of steps.
- It then decays with a cosine curve to almost zero, about max_lr / 250000.
- It also cycles Adam's beta1 between 0.95 and 0.85, opposite to the learning rate.

```
77      scaler = torch.amp.GradScaler("cuda", enabled=use_amp)
```

The gradient scaler for mixed precision. It multiplies the loss by a large factor before backpropagation so small float16 gradients don't underflow to zero. With enabled=False (CPU or MPS), it does nothing and passes everything through.

```
79      ckpt_dir = config.ROOT / args.ckpt_dir
80      ckpt_dir.mkdir(exist_ok=True)
81      ckpt_path = ckpt_dir / f"{args.arch}_best.pt"
```

- **Line 79** builds the absolute checkpoint folder path inside the project folder.
- **Line 80** creates it if needed. Note that train.py uses args.ckpt_dir here, not config.CKPT_DIR.
- **Line 81** builds the file path, for example checkpoints/resnet18_best.pt.

```
82      history, best_val, bad = [], float("inf"), 0
```

Three variables assigned at once:

- history is an empty list that will hold one dictionary per epoch.
- best_val starts at infinity, so the first epoch always counts as an improvement.
- bad counts epochs without improvement.

Part 6: The training loop (lines 84–122)

```
84    print(f"\n{'epoch':>5} {'tr_loss':>8} {'tr_acc':>7} {'va_loss':>8} {'va_acc':>7} {'time':>6}")
```

Prints the column headers. :>5 right-aligns each text in a field 5 characters wide, and so on, so the headers line up with the rows printed later.

```
85    for epoch in range(1, args.epochs + 1):
```

Epoch numbers 1 to 20, so the display starts at 1.

```
86        t0 = time.time()
87        model.train()
```

- **Line 86** records the start time.
- **Line 87** switches back to training mode, which re-enables BatchNorm batch statistics and dropout. It is required because evaluate set the model to eval mode at the end of the previous epoch.

```
88        perm = torch.randperm(len(Xtr)).to(device)
```

Creates a random ordering of the indices 0 to N−1 and moves it to the device. Each epoch sees the data in a different order.

```
89        tl, tc, seen = 0.0, 0, 0
```

Per-epoch accumulators: total loss, correct predictions and samples seen.

```
90        for i in range(0, len(Xtr), args.batch):
91            idx = perm[i:i + args.batch]
```

- **Line 90** steps through the shuffled list in strides of 64.
- **Line 91** takes the next 64 shuffled indices (fewer for the last batch).

```
92            if len(idx) < 2:
93                continue
```

If the final batch has only 1 sample, skip it. BatchNorm cannot compute statistics from a single value, so it would raise an error in training mode.

```
94            xb = augment(Xtr[idx].unsqueeze(1).float())
```

Step by step:

- Xtr[idx] picks the batch, shape (64, 128, 128), float16.
- .unsqueeze(1) adds the channel axis, giving (64, 1, 128, 128).
- .float() converts to float32.

- augment(...) applies the random shift and mask from Part 3.

```
95          yb = ytr[idx]
```

The labels for the same samples, in the same order.

```
96          opt.zero_grad(set_to_none=True)
```

Clears the gradients from the previous step. PyTorch accumulates gradients by default. set_to_none=True frees the gradient tensors rather than filling them with zeros, which is slightly faster.

```
97          with torch.autocast(device_type="cuda", enabled=use_amp):
```

Inside this block on CUDA, operations such as convolutions run in float16 for speed, while sensitive ones stay in float32. On CPU or MPS (enabled=False) the block does nothing.

```
98              out = model(xb).squeeze(1)
```

The forward pass: (64, 1, 128, 128) → (64, 1) logits → squeeze(1) → (64,).

```
99              loss = loss_fn(out.float(), yb)
```

Converts the logits to float32 and computes the mean binary cross-entropy between the logits and labels. Doing the loss in float32 avoids precision problems.

```
100         scaler.scale(loss).backward()
```

Scales the loss up (on CUDA) and backpropagates, filling .grad on every parameter.

```
101         scaler.step(opt)
```

Un-scales the gradients and calls opt.step() to update the weights. If any gradient contains infinity or NaN, which can happen with float16, the update is skipped for that batch.

```
102         scaler.update()
```

Adjusts the scale factor for the next batch. It grows when things are stable and shrinks after an overflow.

```
103         sched.step()
```

Advances the learning-rate schedule by one step. Since OneCycleLR is designed to be stepped per batch, this happens inside the inner loop.

```
104         tl += loss.item() * len(idx)
105         tc += ((out > 0).float() == yb).sum().item()
106         seen += len(idx)
```

Bookkeeping, using the same technique as in evaluate:

- **Line 104** adds the batch loss multiplied by the batch size.
- **Line 105** counts the correct predictions.
- **Line 106** counts the samples processed.

These are measured on augmented data in training mode, so the training numbers are not directly comparable with the validation numbers.

```
108     va_loss, va_acc = evaluate(model, Xva, yva, loss_fn)
```

After the epoch's batches are done, the model is run on the full validation set without augmentation. The results are the real indicator of generalisation.

```
109     row = dict(epoch=epoch, train_loss=tl / seen, train_acc=tc / seen, val_loss=va_loss,
val_acc=va_acc)
110     history.append(row)
```

Builds one record with the epoch's averages and appends it to history.

```
111     print(f"{epoch:>5} {row['train_loss']:>8.4f} {row['train_acc']:>7.3f} "
112           f"{va_loss:>8.4f} {va_acc:>7.3f} {time.time()-t0:>5.0f}s")
```

Prints one table row. >8.4f means right-aligned in 8 characters with 4 decimals. The last field is the elapsed time in whole seconds.

Checkpointing and early stopping

```
114     if va_loss < best_val:
115         best_val, bad = va_loss, 0
```

If this epoch's validation loss is the best so far, record it as the new best and reset the patience counter.

```
116         torch.save({"arch": args.arch, "state_dict": model.state_dict(),
117                     "epoch": epoch, "val_loss": va_loss, "val_acc": va_acc}, ckpt_path)
```

Saves a dictionary to the checkpoint file, overwriting the previous best. It contains the architecture name, the weights (state_dict), and the epoch and validation metrics. infer.py and the GUI read this dictionary back.

```
118     else:
119         bad += 1
```

No improvement this epoch, so add one to the counter.

```
120         if bad >= args.patience:
121             print(f"Early stopping (no val-loss improvement for {args.patience}
epochs).")
122             break
```

After 5 epochs in a row without improvement, print a message and exit the epoch loop early. This prevents overfitting and saves time.

Part 7: Wrap-up (lines 124–128)

```
124     with open(config.RESULTS_DIR / f"history_{args.arch}.json", "w") as f:
125         json.dump(history, f, indent=2)
```

Opens results/history_resnet18.json for writing (the with block closes the file automatically) and writes the list of per-epoch dictionaries as indented JSON. This is the data behind training curves.

```
126     best = min(history, key=lambda r: r["val_loss"])
```

Finds the history row with the lowest validation loss. key=lambda r: ... tells min to compare rows by that field. This is the same epoch that was last saved as the checkpoint.

```
127     print(f"\nBest epoch: {best['epoch']} | val loss {best['val_loss']:.4f} | val acc
{best['val_acc']:.3f}")
128     print(f"Checkpoint: {ckpt_path}")
```

Prints the summary and where the model was saved.

Part 8: Entry point (lines 131–132)

```
131 if __name__ == "__main__":
132     main()
```

When you run python train.py, Python sets __name__ to "__main__", so main() runs. When another file imports train.py, as train_mixed.py does with from train import augment, __name__ is "train", so the imports and function definitions run but training does **not** start.

Summary of one training step

```
Xtr[idx] (64,128,128) fp16 → unsqueeze/float → augment (shift + mask)
  → model → logits (64,) → BCE loss vs labels
  → backward → AdamW update → OneCycleLR step
```

This repeats about 110 times per epoch, then the validation pass and the checkpoint decision follow.

I can write the same kind of line-by-line explanation for sonar_model.py, sonar_config.py, simulator.py or train_mixed.py if you'd like. I can also put this into a Word or PDF document for your report.